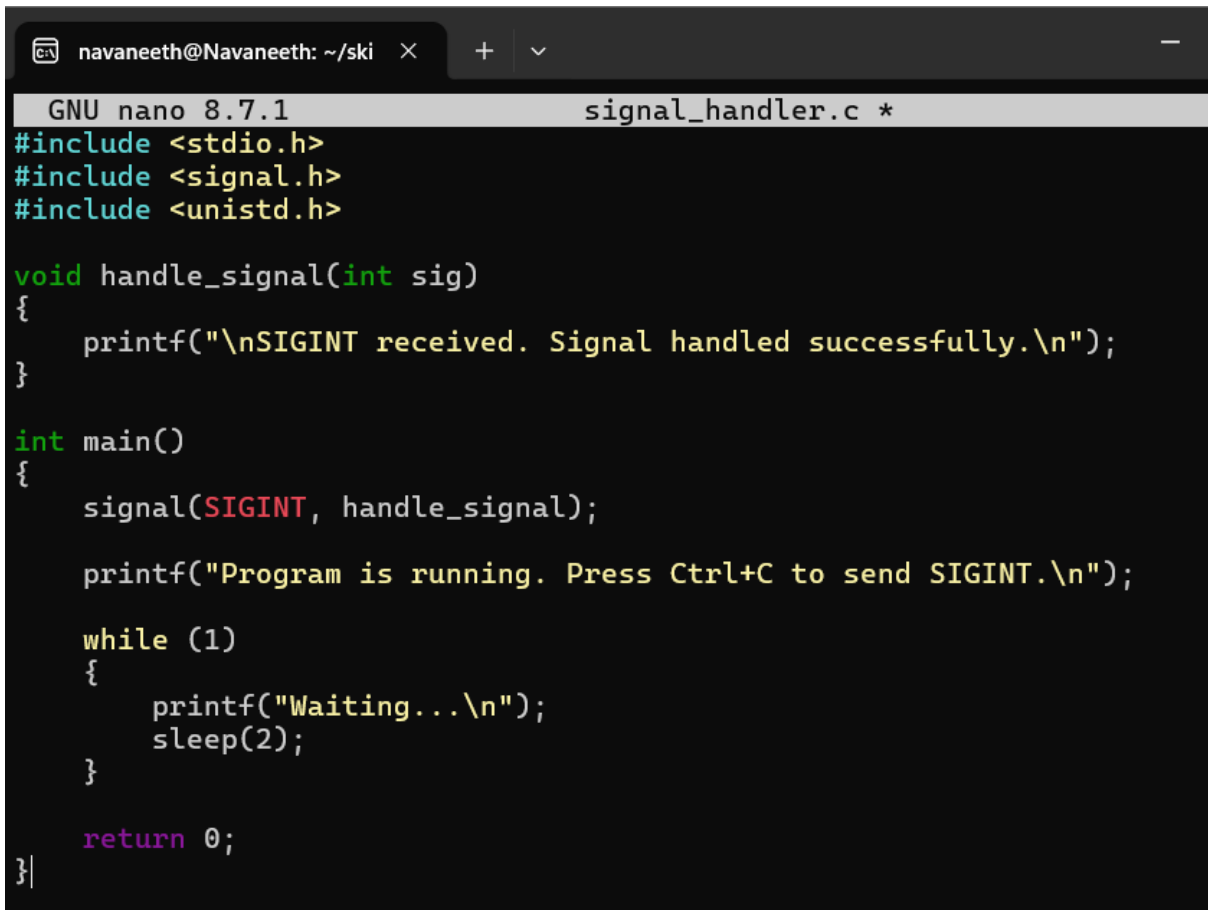


Skill Week 10

Aim

To learn how to handle Linux signals by registering a signal handler, responding to an interrupt such as SIGINT, and verifying the signal-handling behavior using a simple C program.

Program:



```
navaneeth@Navaneeth: ~/ski × + ▾
GNU nano 8.7.1 signal_handler.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handle_signal(int sig)
{
    printf("\nSIGINT received. Signal handled successfully.\n");
}

int main()
{
    signal(SIGINT, handle_signal);

    printf("Program is running. Press Ctrl+C to send SIGINT.\n");

    while (1)
    {
        printf("Waiting...\n");
        sleep(2);
    }

    return 0;
}
```

Output:

```
navaneeth@Navaneeth: ~/ski x + v
navaneeth@Navaneeth:~$ mkdir -p skillweek10
navaneeth@Navaneeth:~$ cd skillweek10
navaneeth@Navaneeth:~/skillweek10$ nano signal_handler.c
navaneeth@Navaneeth:~/skillweek10$ gcc signal_handler.c -o signal_handler
navaneeth@Navaneeth:~/skillweek10$ ./signal_handler
Program is running. Press Ctrl+C to send SIGINT.
Waiting...
Waiting...
Waiting...
Waiting...
Waiting...
^C
SIGINT received. Signal handled successfully.
Waiting...
Waiting...
^Z
[1]+  Stopped                  ./signal_handler
navaneeth@Navaneeth:~/skillweek10$ jobs
[1]+  Stopped                  ./signal_handler
navaneeth@Navaneeth:~/skillweek10$ kill %1

[1]+  Stopped                  ./signal_handler
[1]+  Terminated              ./signal_handler
navaneeth@Navaneeth:~/skillweek10$ |
```

Observation

The signal() function registered a handler for SIGINT. When Ctrl+C was pressed, the signal handler executed and displayed a message instead of allowing the default termination behavior.

Result

A Linux **signal handler** was **successfully registered, tested, and verified** using a C program and SIGINT.

Aim

To understand how SIGINT is generated by **Ctrl+C**, how signals affect foreground processes, and how a process can handle SIGINT without immediately terminating.

Program:

```
navaneeth@Navaneeth: ~/ski × + v
GNU nano 8.7.1 sigint_test.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handle_sigint(int sig)
{
    printf("\nSIGINT received. Process is still running.\n");
}

int main()
{
    signal(SIGINT, handle_sigint);

    printf("Process started. Press Ctrl+C to send SIGINT.\n");

    while (1)
    {
        printf("Running...\n");
        sleep(2);
    }

    return 0;
}
```

Output:

```
navaneeth@Navaneeth:~/skillweek10$ nano sigint_test.c
navaneeth@Navaneeth:~/skillweek10$ gcc sigint_test.c -o sigint_test
navaneeth@Navaneeth:~/skillweek10$ ./sigint_test
Process started. Press Ctrl+C to send SIGINT.
Running...
Running...
^C
SIGINT received. Process is still running.
Running...
Running...
^Z
[1]+  Stopped                  ./sigint_test
navaneeth@Navaneeth:~/skillweek10$ kill %1

[1]+  Stopped                  ./sigint_test
[1]+  Terminated              ./sigint_test
navaneeth@Navaneeth:~/skillweek10$ jobs
navaneeth@Navaneeth:~/skillweek10$
```

Observation

Ctrl+C sends SIGINT to the foreground process. The registered signal handler catches the signal and allows the program to continue. Ctrl+Z stops the foreground process and places it in the job table.

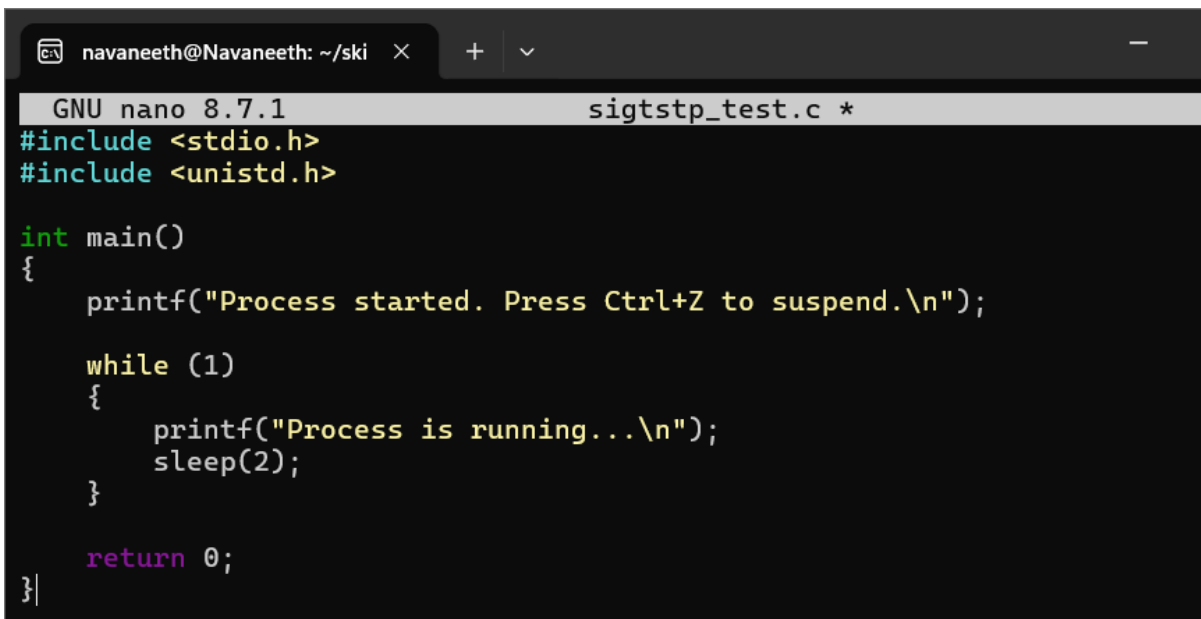
Result

SIGINT handling, foreground-process interruption, job-state updating, and process termination were successfully tested.

Aim

To understand how SIGTSTP suspends a foreground process, how the suspended process appears in the job table, and how to resume the process later.

Program:



```
navaneeth@Navaneeth: ~/ski  ×  +  ▾  —
GNU nano 8.7.1 sigtstp_test.c *
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Process started. Press Ctrl+Z to suspend.\n");

    while (1)
    {
        printf("Process is running...\n");
        sleep(2);
    }

    return 0;
}
```

Output:

```

navaneeth@Navaneeth:~/skillweek10$ nano sigtstp_test.c
navaneeth@Navaneeth:~/skillweek10$ gcc sigtstp_test.c -o sigtstp_test
navaneeth@Navaneeth:~/skillweek10$ ./sigtstp_test
Process started. Press Ctrl+Z to suspend.
Process is running...
Process is running...
^Z
[1]+  Stopped                  ./sigtstp_test
navaneeth@Navaneeth:~/skillweek10$ jobs
[1]+  Stopped                  ./sigtstp_test
navaneeth@Navaneeth:~/skillweek10$ fg %1
./sigtstp_test
Process is running...
Process is running...
^Z
[1]+  Stopped                  ./sigtstp_test
navaneeth@Navaneeth:~/skillweek10$ jobs
[1]+  Stopped                  ./sigtstp_test
navaneeth@Navaneeth:~/skillweek10$ kill %1

[1]+  Stopped                  ./sigtstp_test
[1]+  Terminated              ./sigtstp_test
navaneeth@Navaneeth:~/skillweek10$ jobs
navaneeth@Navaneeth:~/skillweek10$ |

```

Observation

Ctrl+Z sends SIGTSTP to the foreground process and suspends it. The jobs command shows the process as **Stopped**. The fg command resumes it in the foreground, while bg resumes it in the background.

Result

The **SIGTSTP suspension and process-resume workflow** was successfully implemented and verified using Linux job-control commands.

Aim

To understand **process groups and terminal control** in Linux by creating a background process, checking its process group, transferring it to the foreground, and observing its execution.

```
navaneeth@Navaneeth:~/skillweek10$ sleep 60 &
[1] 747
navaneeth@Navaneeth:~/skillweek10$ jobs -l
[1]+  747 Running                  sleep 60 &
navaneeth@Navaneeth:~/skillweek10$ ps -o pid,ppid,pgid,sid,tt,stat,cmd
  PID   PPID   PGID   SID TT   STAT CMD
   327    326    327    327 pts/0  Ss   -bash
   747    327    747    327 pts/0  S    sleep 60
   766    327    766    327 pts/0  R+   ps -o pid,ppid,pgid,sid,tt,stat,cmd
navaneeth@Navaneeth:~/skillweek10$ fg %1
sleep 60
navaneeth@Navaneeth:~/skillweek10$ jobs
navaneeth@Navaneeth:~/skillweek10$ sleep 30 &
[1] 800
navaneeth@Navaneeth:~/skillweek10$ ps -o pid,ppid,pgid,sid,tt,stat,cmd
  PID   PPID   PGID   SID TT   STAT CMD
   327    326    327    327 pts/0  Ss   -bash
   800    327    800    327 pts/0  S    sleep 30
   810    327    810    327 pts/0  R+   ps -o pid,ppid,pgid,sid,tt,stat,cmd
navaneeth@Navaneeth:~/skillweek10$ fg %1
sleep 30
navaneeth@Navaneeth:~/skillweek10$ jobs
navaneeth@Navaneeth:~/skillweek10$ |
```

Observation

The `ps` command displays the process ID, parent process ID, process group ID, session ID, terminal, and process state. The `fg` command transfers a background job to the foreground and gives it terminal control.

Result

Process groups and **terminal ownership/control** were successfully observed and verified using Linux process and job-control commands.